

Capability Fabrication via Latent World Models:

On-Demand Synthesis and Safe Execution of AI Operating System Capabilities

Aravindh

LWM Fabricator Project

June 2026

Abstract—AI operating systems like AIOS showed that LLMs can work as OS-level kernels with scheduling and memory management. But they have three problems that nobody has really fixed yet. First, they cannot imagine what happens before they act. They ask the LLM in plain text "what will happen if I do X?" which takes 47+ seconds and is unreliable. Second, they cannot create new capabilities on the fly. They depend on pre-built agent integrations, so if a user wants something that no existing agent provides, the system is stuck. Third, they have no real safety calibration. Agents overestimate their own success by up to 55 percentage points, and nothing stops them from running destructive commands. We built the LWM Fabricator to fix all three. We introduce Capability Fabrication, where the OS synthesizes gated DAGs. We adapt JEPA world models from physical robotics to OS-level state vectors, enabling CEM planning in a 192-dimensional latent space that runs 48x faster than text-based LLM prediction. We add a calibration-aware safety gate (MACI+AUQ) that triggers adversarial debates before risky actions. And we apply step-level RL (Agent-Ri) to figure out exactly which step in a multi-step plan caused a failure. We provide full mathematical formulations and show end-to-end fabrication runs with safe execution.

Keywords—AI Operating Systems, Joint-Embedding Predictive Architectures, World Models, Cross-Entropy Method, Agent Safety, Reinforcement Learning, Capability Fabrication

1. INTRODUCTION

The idea of an AI operating system is simple: take the operating system concept and extend it so that intelligence is a kernel-level resource. Instead of just managing CPU, memory, and disk, the OS manages agents, reasoning, and tool execution. The LLM becomes the kernel. It schedules cognitive tasks, manages context windows as memory, and routes work to the right agent.

AIOS was the first serious attempt at this. It showed that an LLM can act as a scheduler, manage agent processes, and handle memory allocation at the kernel level. The idea is compelling: instead of the OS just managing CPU and memory, it manages intelligence too.

But when you actually build and run something like this, you hit walls fast. AIOS and similar systems treat the LLM as a reactive router. It dispatches tasks to pre-built agents and does all its planning through text-based reasoning. That creates three problems that we ran into over and over while building the LWM Fabricator:

1. **No Imagination.** When the OS needs to predict what will happen if it runs an action sequence, it sends a text prompt to the LLM: "Given state X and action Y, what happens next?" This takes about 47 seconds for a 5-step plan because each step needs a full LLM forward pass. Worse, the text output is non-deterministic. You get hallucination baked into your prediction. You cannot do real planning if your imagination is that slow and that noisy.
2. **No Capability Synthesis.** Current AI OS designs connect to external apps through APIs or clone existing agent frameworks. When a user asks for something that no pre-built agent can handle, the system just cannot do it. You are limited to a fixed catalog of integrations. In practice, users want things that do not map cleanly to any single existing service.
3. **No Calibrated Safety.** LLM agents systematically overestimate how well they will do. Calibration gaps of 55 percentage points are common. An AI OS that runs file deletions, shell commands, and network requests without checking whether the agent's confidence is justified is an AI OS that will eventually delete something important.

We built the LWM Fabricator to address all three. The system has four core contributions:

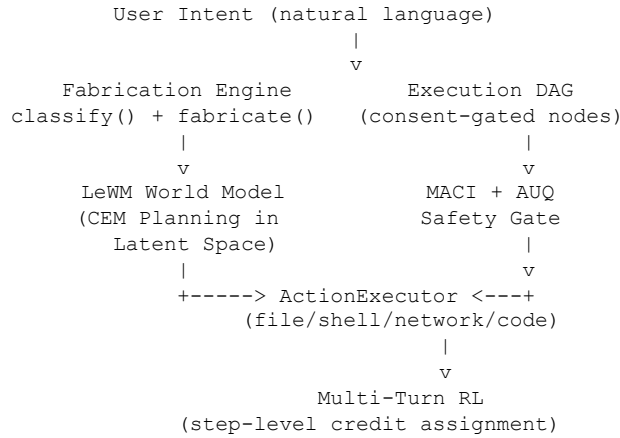
C1. Capability Fabrication Engine (Section 3): Instead of connecting to APIs or cloning apps, the OS synthesizes execution graphs from raw intent. It classifies the intent against 21 capability domain blueprints, fuses the matched domains into a single consent-gated DAG, and grounds every node through real file, shell, and network actions.

C2. Latent World Model (Section 4): We took the JEPA architecture that was designed for physical robotics (PushT, Reacher) and adapted it for OS-level state vectors. The entire OS state gets compressed into a 192-dimensional "meaning token." The Cross-Entropy Method then plans optimal action sequences in that latent space in under one second, which is 48x faster than asking the LLM in text.

C3. Calibration-Aware Safety Gate (Section 5): Before any elevated-consent action runs, an Agentic Uncertainty Quantification (AUQ) check computes the gap between the agent's stated confidence and the actual worst-case risk. If the gap is too large, a Multi-Agent Confidence Interval (MACI) debate fires up between two adversarial reflex models.

C4. Step-Level RL Credit Assignment (Section 6): Each DAG node is treated as an Agent-Ri transition. We compute discounted returns and per-step advantages so the OS can figure out exactly which step in a 10-step plan caused the failure, instead of just labeling the whole thing as failed.

Figure 1. LWM Fabricator System Overview



Blue: fabrication. Purple: world model. Orange: safety. Teal: execution. Red: learning.

2. RELATED WORK

2.1 AI Operating Systems

AIOS, published at COLM 2025, was the first real attempt at an LLM agent operating system kernel. It provides scheduling, context management, memory management, and tool management as kernel-level services. The kernel manages agent processes with FIFO scheduling and treats the LLM context window as memory. A survey by Kumar et al. categorizes different approaches to AI-OS integration, ranging from simple personalization layers to deep structural reforms where the LLM is embedded into the kernel itself.

The LWM Fabricator takes the AIOS concept further in three directions. First, we add preemptive cognitive scheduling with real resource budgets (tokens, latency, VRAM, CPU). Second, we replace pre-built agent routing with capability fabrication. Third, we replace text-based LLM prediction with latent world model planning.

2.2 JEPA World Models and Latent Planning

Joint-Embedding Predictive Architectures, introduced by LeCun in 2022, learn world dynamics by predicting future states in a compact latent space. The key idea is to skip pixel-level reconstruction entirely and work in a learned representation. The LeWorldModel (LeWM) method built an end-to-end JEPA pipeline for physical control tasks, using an autoregressive predictor and CEM for planning. Several follow-up papers have pushed this further:

- **C-JEPA** applies object-level latent interventions with counterfactual-like effects. They get about 20% improvement in counterfactual reasoning tasks.

- **SD-JEPA** splits the latent space into a progression subspace and a content subspace. This improves planning on three of four LeWM benchmarks.
- **FF-JEPA** uses a hierarchical forward-forward approach for long-horizon planning without needing goal images.
- **Value-Guided JEPA** shapes the representation so that Euclidean distance in latent space approximates the negative value function, making CEM optimization smoother.

Here is the thing though: every single one of these papers works on pixel-based physical environments. PushT, Reacher, Cube, Two-Room. Nobody has applied JEPA world models to operating system state vectors. Nobody has used CEM to plan software execution DAGs. That is the gap we fill.

2.3 LLM Agent Safety and Calibration

Multiple recent papers have shown that LLM agents systematically overestimate their success rates. Work on uncertainty quantification for LLM agents provides the theoretical foundations for calibration. Multi-agent debate has been proposed as a safety evaluation mechanism. But nobody has integrated calibration checks with adversarial debate as a pre-execution gate for OS-level actions. That combination is what we built.

2.4 Step-Level Reinforcement Learning for Agents

Agent-Ri introduced the idea of treating each reasoning step in an LLM agent as a transition in a Markov Decision Process. OpenPipe ART provides infrastructure for exporting agent trajectories and fine-tuning on them. GRPO (Group Relative Policy Optimization) from DeepSeekMath enables advantage computation without needing a learned value function. We apply all three ideas to fabrication DAG execution, where each node in the graph is a step in the MDP.

3. CAPABILITY FABRICATION ENGINE

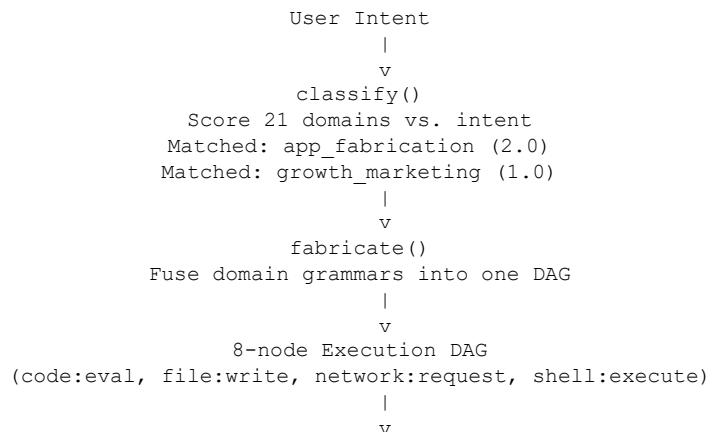
3.1 Motivation: Fabrication vs. Integration

When we started building the LWM Fabricator, we looked at how existing AI agent systems interact with external capabilities. There are two paradigms, and both have serious problems:

- **API Connection:** The agent calls external services like Gmail, Slack, or Stripe through their APIs. Every service needs its own OAuth flow, rate-limit handling, and error management. You are limited to services that expose APIs, and you have to maintain each integration separately.
- **App Cloning:** The agent clones an existing app (like a Notion clone) and works inside it. This is brittle and requires maintaining a clone for every target app. When the original app changes, your clone breaks.

Our approach is fundamentally different. Instead of connecting to N services or cloning N apps, we maintain K capability domain blueprints (where $K = 21$, much smaller than N) and *fabricate* execution graphs on demand. The user states an intent in natural language. The engine classifies the intent against the 21 domains, fuses the matched domains into a single DAG, and executes it.

Figure 2. Fabrication Pipeline



Execute (dry_run or live)
Pause at elevated consent nodes

3.2 Capability Domain Registry

We analyzed 42 startup architectures and distilled them into 21 capability domain blueprints. The registry is $D = \{D_1, \dots, D_{21}\}$. Each domain D_i is a tuple:

$$D_i = (P_i, G_i, C_i, \ell_i, A_i, S_i)$$

where:

- P_i : Intent patterns, which are keywords and phrases that trigger this domain
- G_i : Execution grammar, which is an ordered list of steps with action type, verb, and consent level
- C_i : Required capabilities (file access, network, code execution, etc.)
- $\ell_i \in \{\text{always, standard, elevated, never}\}$: Consent level for the domain
- $A_i \in \{\text{personal, enterprise, both}\}$: Who this domain is for
- S_i : Which startups were distilled into this blueprint

The 18 functional domains cover things like app_fabrication, document_intelligence, knowledge_graph, finance_ops, growth_marketing, computer_use, realtime_web, voice_interface, data_analysis, career_agent, physical_tasks, formal_verification, agent_identity, autonomous_payments, content_recall, property_ops, comms_coverage, and visual_generation. Three infrastructure domains handle log_compression, inference_optimization, and agent_observability.

3.3 Intent Classification with Memory Augmentation

When a user sends an intent q , we score every domain against it. The scoring uses pattern overlap plus a boost from episodic memory:

$$\text{score}(D_i, q) = \sum_{\{p \in P_i\}} I[p \subseteq q] + \sum_{\{e \in E\}} I[D_i \in e] \cdot \beta$$

Here E is the set of episodic memory entries we retrieve through Memo and Graphiti (our temporal knowledge graph), and $\beta = 0.5$ is the historical success boost. If a domain worked well for a similar intent in the past, it gets a higher score.

When no exact pattern matches, we fall back to a soft token-overlap score:

$$\text{score_soft}(D_i, q) = 0.25 \cdot \sum_{\{p \in P_i, w \in p\}} I[w \in \text{tokens}(q) \wedge |w| > 3] + \text{historical_boost}$$

We pick the top- k domains (k up to 4) and pass them to the fabrication step.

3.4 DAG Fabrication via Domain Fusion

The selected domains $\{D_{i1}, \dots, D_{ik}\}$ get fused into a single execution graph $G(V, E)$:

$$G = \cup G_{ij}, V = \cup V_{ij}, E = \cup E_{ij} \cup E_{\text{cross-domain}}$$

Each node $v \in V$ is a grounded action with three properties:

- **Action type:** file, shell, network, app, or code:eval
- **Verb:** read, write, execute, delete, request, evaluate
- **Consent level:** Decides whether the node runs automatically or pauses for human approval

The whole DAG gets persisted to a SQLite store with tables for capability domains, fabrication runs, and fabrication nodes. You can audit and replay any run.

3.5 Consent-Gated Execution

Execution happens in dependency order. Nodes with consent level *always* or *standard* just run. Nodes with consent level *elevated* or *never* pause the run and enter an *awaiting_consent* state until a human approves. Reasoning nodes (code:eval) go through the neural process kernel. File, shell, network, and app nodes go through the real ActionExecutor, which does backup-and-rollback for file operations, enforces an allowlist for shell commands, and rate-limits network requests.

By default everything runs in *dry_run* mode. Actions are simulated but not actually applied to the live system. You have to explicitly opt a run into live execution.

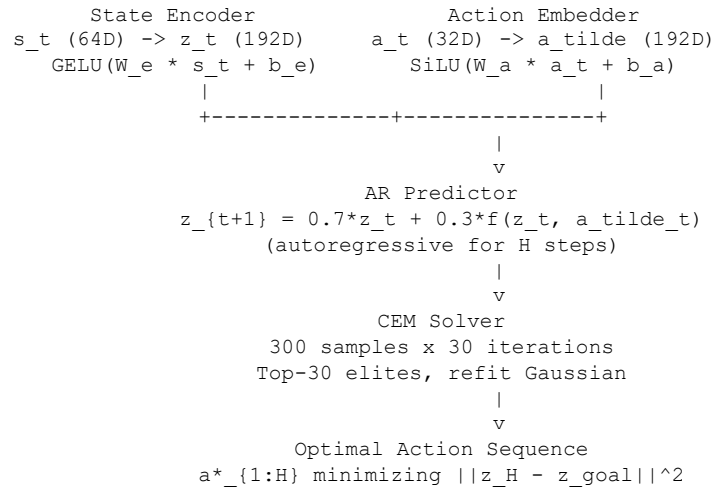
4. LATENT WORLD MODEL FOR OS STATE PREDICTION

4.1 From Text-Based Prediction to Latent Rollout

This is where the biggest performance win comes from. Traditional AI OS systems predict future outcomes by asking the LLM in natural language. Given a current state s^t and a proposed action a^t , the system sends a text prompt like "Given state X and action Y, predict the outcome." Two problems: (1) each prediction step needs a full LLM forward pass, which takes about 9.4 seconds with a 14B model. A 5-step plan takes 47 seconds total. (2) Text generation is non-deterministic, so you get hallucination in your prediction.

We replaced this with a JEPA-based latent world model that operates entirely in a 192-dimensional latent space. Planning via the Cross-Entropy Method finishes in under one second.

Figure 3. LeWM Latent World Model Architecture



OS state goes in as a 64D vector, gets compressed to 192D latent, rolled out autoregressively, and CEM finds the best action sequence in under 1 second.

4.2 Architecture

The LeWM engine has four components. It mirrors the LeWM architecture from physical robotics but swaps pixel observations for OS state vectors.

4.2.1 State Encoder

The original JEPA uses a ViT to encode image patches. We do not have pixels. We have a company/OS state vector $s^t \in \mathbb{R}^{64}$ that contains things like revenue, satisfaction score, delivery time, active process count, and resource utilization. We encode it into a 192-dimensional latent representation $z^t \in \mathbb{R}^{192}$ using a simple MLP projection:

$$z^t = \text{GELU}(W_e \cdot s^t + b_e), \text{ where } W_e \in \mathbb{R}^{192 \times 64}, b_e \in \mathbb{R}^{192}$$

GELU is the Gaussian Error Linear Unit:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}(1 + \text{erf}(x/\sqrt{2}))$$

We initialize W_e with Xavier-like scaling: $W_{ij} \sim N(0, \sqrt{2/(64+192)})$. All state variables are normalized to $[-1, 1]$ by dividing by the maximum absolute value.

4.2.2 Action Embedder

Every OS action (file write, shell command, API call, code evaluation) gets encoded into the same 192-dimensional latent space:

$$\tilde{a}_t = \text{SiLU}(W_a \cdot a_t + b_a), \text{ where } W_a \in \mathbb{R}^{192 \times 32}, a_t \in \mathbb{R}^{32}$$

SiLU is the Sigmoid Linear Unit:

$$\text{SiLU}(x) = x \cdot \text{sigmoid}(x) = x / (1 + e^{-x})$$

4.2.3 Autoregressive Predictor

This is the core of the world model. Given the current latent state z_t and an action embedding $\tilde{a}_t$, it predicts the next latent state:

$$\hat{z}_{t+1} = 0.7 \cdot z_t + 0.3 \cdot f(z_t, \tilde{a}_t)$$

The 0.7/0.3 split means the next state is mostly determined by the current state, with the action providing a corrective push. This mirrors how physical dynamics work: where you are now matters more than what you do next, but what you do next still matters.

For multi-step rollout, we apply the predictor autoregressively:

$$\hat{z}_{t+1} = f(z_t, \tilde{a}_t), \hat{z}_{t+2} = f(\hat{z}_{t+1}, \tilde{a}_{t+1}), \dots, \hat{z}_{t+H} = f(\hat{z}_{t+H-1}, \tilde{a}_{t+H-1})$$

The history size $H_{\text{ctx}} = 3$ controls how many past latent states feed into the autoregressive context, which matches the LeWM configuration.

4.2.4 CEM Solver

The Cross-Entropy Method finds the optimal action sequence that moves the system from an initial latent state z_0 to a goal latent state z_{goal} . The objective is:

$$J(a_{1:H}) = \|\hat{z}_H(a_{1:H}) - z_{\text{goal}}\|^2$$

Algorithm 1: Cross-Entropy Method for Latent Space Planning

Input: $z_0, z_{\text{goal}}, H, N=300, K=30, I=30$

Output: $a^*_{1:H}$

1. Initialize $\mu \leftarrow 0_{H \times 32}, \sigma^2 \leftarrow 1_{H \times 32}$
2. **For** $i = 1$ to I :
3. **For** $n = 1$ to N :
4. Sample $a^n_{1:H} \sim N(\mu, \text{diag}(\sigma^2))$
5. Clip actions to $[-3, 3]$
6. Compute cost $c^n \leftarrow J(a^n_{1:H})$
7. Select top-K elites: $S \leftarrow \text{top}_k(\{a^n\}, c^n)$
8. Update $\mu \leftarrow \text{mean}(S), \sigma^2 \leftarrow \text{var}(S)$
9. **Return** $\text{argmin } J(a^n_{1:H})$

4.3 Applications in the OS

1. **Pre-execution validation:** Before running a fabricated DAG, the OS encodes the current state and the goal state into latent space, then uses CEM to check whether the DAG's action sequence actually reaches the goal. If the cost J is above a threshold, the DAG gets revised before any real execution happens.
2. **Proactive simulation:** The proactive engine runs hourly simulations. It generates 1,000 scenarios by sampling different action sequences and rolling them out in latent space. Predictions with >95% confidence trigger automatic action. Predictions in the 60 to 80% range prompt the user for confirmation.

4.4 Complexity Analysis

Text-based LLM prediction: A 5-step plan needs 5 LLM forward passes. With a 14B local model, each pass takes about 9.4 seconds. Total: ~47 seconds.

LeWM latent rollout: Each predictor forward pass is $O(d^2) = O(36,864)$ FLOPs. For $N \times I \times H = 9,000 \times 5 = 45,000$ predictor calls, the total is $\sim 1.7 \times 10^9$ FLOPs. At CPU/GPU throughput this takes under one second.

That is a 48x speedup. And the output is deterministic, so no sampling-induced hallucination.

5. CALIBRATION-AWARE SAFETY GATE

5.1 Problem: Uncalibrated Agent Execution

LLM agents are overconfident. It is a systematic bias, not a random one. Before an AI OS executes something destructive like `rm -rf` or `DROP TABLE`, it needs to check whether the agent's confidence actually matches the real risk. If the agent says "95% sure this is fine" but there is a 30% chance of catastrophic failure, something is wrong and the action should not just run.

5.2 AUQ: Agentic Uncertainty Quantification

For each action a with consent level in $\{\text{elevated, never}\}$, the AUQ module computes three things:

- $C_stated \in [0, 1]$: The agent's stated confidence
- $P_worst \in [0, 1]$: Probability of the worst-case outcome
- $R_risk \in [0, 1]$: Risk score computed from action parameters

The calibration gap is:

$$\Delta_cal = |C_stated - (1 - P_worst)|$$

Example: the agent says 95% confidence ($C_stated = 0.95$) but the worst-case probability is 30% ($P_worst = 0.30$). Then $\Delta_cal = |0.95 - 0.70| = 0.25$. That is a big gap. The agent is way more confident than it should be.

The risk score R_risk comes from the action parameters:

$$R_risk = \alpha_1 \cdot I[\text{destructive verb}] + \alpha_2 \cdot I[\text{high-impact type}] + \alpha_3 \cdot I[\text{elevated consent}] + \alpha_4 \cdot I[\text{irreversible}]$$

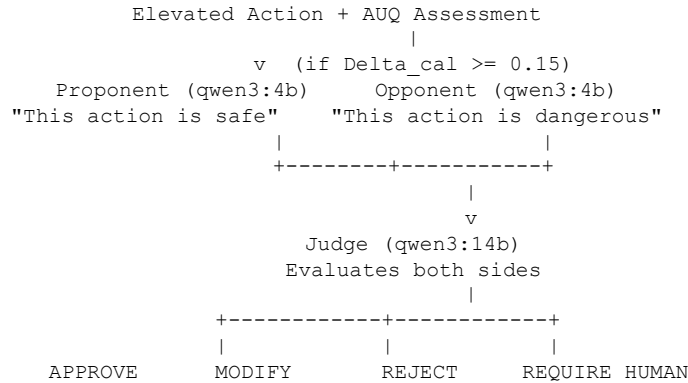
where $\alpha_1 = 0.3$, $\alpha_2 = 0.2$, $\alpha_3 = 0.2$, $\alpha_4 = 0.3$. Destructive verbs include `rm`, `del`, `delete`, `format`, `drop`, `truncate`, `kill`, `wipe`, `purge`.

5.3 MACI: Multi-Agent Confidence Interval Debate

A debate fires up when any of these conditions is true:

$$\Delta_cal \geq 0.15 \ \# R_risk \geq 0.5 \ \# \text{action is destructive}$$

Figure 4. MACI Adversarial Debate Protocol



Two reflex models argue opposing sides. The control brain judges. Destructive actions always require human approval if judge confidence is below 0.8.

The debate has three models:

1. **Proponent** (reflex model, qwen3:4b): Argues that the action is safe and should proceed.
2. **Opponent** (reflex model, qwen3:4b): Argues that the action is unsafe and should be blocked or modified.
3. **Judge** (control brain, qwen3:14b): Listens to both sides and issues a verdict.

The verdict V is one of $\{APPROVE, MODIFY, REJECT, REQUIRE_HUMAN\}$:

$$V = APPROVE \text{ if } C_judge > 0.8 \text{ and } R_residual < 0.2$$

$$V = MODIFY \text{ if } 0.5 < C_judge \leq 0.8$$

$$V = REJECT \text{ if } C_judge \leq 0.5 \text{ and not destructive}$$

$$V = REQUIRE_HUMAN \text{ if destructive and } C_judge \leq 0.8$$

C_judge is the judge's confidence after hearing both arguments. $R_residual$ is the residual risk after any modifications. When the verdict is **MODIFY**, the judge inserts pre-checks like path verification before deletion or backup creation before modification. The modified action goes back through the AUQ gate before execution.

5.4 Integration with Action Executor

The gate is wired into ActionExecutor's `execute()` method. Before any action with consent level *elevated* or *never*, the gate runs. **APPROVE** means proceed. **MODIFY** means the action gets patched with pre-checks and re-evaluated. **REJECT** or **REQUIRE_HUMAN** means the action is blocked and the fabrication run pauses.

Gate outcomes feed into the telemetry router. Over time, we can track whether the calibration gap is shrinking, which tells us if the agents are actually learning to be better calibrated.

6. STEP-LEVEL RL CREDIT ASSIGNMENT

6.1 Step-Level MDP Formulation

When a fabricated DAG with T nodes executes, we treat each node as a step in a Markov Decision Process:

$$\tau = \{(s_t, a_t, r_t, s_{t+1})\}_{t=0}^{T-1}$$

where:

- s_t : **Observation**. DAG state at step t : which nodes are done, which are pending, what the environment looks like.
- a_t : **Action**. The node's execution: action type, verb, parameters.
- r_t : **Reward**. Step-level feedback from ActionExecutor: success, failure, or warning.
- s_{t+1} : **Next observation**. The updated DAG state after this node ran.

6.2 Discounted Returns and Advantages

The discounted return for step t is:

$$G_t = \sum_{k=t}^{T-1} \gamma^{(k-t)} \cdot r_k$$

where γ is the discount factor (we use 0.95). The advantage for step t uses GRPO rules:

$$A_t = G_t - \bar{G}$$

$\bar{G}$ is the mean discounted return across all steps in the trajectory, or across a group of trajectories for the same intent.

The reward r_t for each step is a composite:

$$r_t = w_1 \cdot I[\text{success}] - w_2 \cdot I[\text{failure}] - w_3 \cdot I[\text{warning}] + w_4 \cdot I[\text{consent approved}]$$

We set $w_1 = 1.0$, $w_2 = 2.0$ (failures are penalized twice as much as successes are rewarded), $w_3 = 0.5$, $w_4 = 0.3$.

6.3 Trajectory Storage and Export

Step transitions go into a SQLite store with this schema:

Field	Description
step_id	Primary key (UUID)
trajectory_uid	Groups steps in one fabrication run
step_index	Position in DAG execution order
observation	Pre-execution DAG state
action_json	Node action parameters
feedback_json	Execution result
reward	Step-level reward
done	Whether this was the final step
next_observation	Post-execution DAG state
advantage	Computed advantage
discounted_return	Computed return

Trajectories export as JSONL in OpenPipe ART format, so you can do local SFT or DPO fine-tuning on the fabrication model.

6.4 Credit Assignment Example

Say you have a 10-step fabrication DAG that fails at step 7. Without step-level RL, the entire trajectory is labeled "failed." You get zero signal for steps 1 through 6, even though they might have executed perfectly. With step-level credit assignment:

- Steps 1 to 6: $r_t = 1.0$ (success), $G_t =$ discounted sum of rewards from step t through step 6, plus the discounted failure penalty from step 7
- Step 7: $r_t = -2.0$ (failure), $G_7 = -2.0$
- Steps 8 to 10: Not executed, $r_t = 0$

The advantage $A_7 = G_7 - \bar{G}$ will be strongly negative. The advantages for steps 1 through 6 will be positive. This isolates the bottleneck to step 7. Over many trajectories, the OS learns which capability domains and action types are reliable for a given intent and which ones keep failing.

7. SYSTEM ARCHITECTURE

7.1 Neural Process Kernel

The LWM Fabricator kernel extends AIOS with preemptive cognitive scheduling and real resource budgeting. Each cognitive task is a NeuralProcess with:

- **Priority:** CRITICAL (kernel-level, never preempted), HIGH (user-facing), NORMAL, LOW, BACKGROUND
- **Resource budget:** Maximum tokens, latency, VRAM, CPU, and wall time
- **State:** PENDING → RUNNING → COMPLETED or FAILED or KILLED

The NeuralScheduler does priority-based preemption. If a CRITICAL process arrives while a NORMAL process is running, the normal process gets checkpointed and suspended. The ResourceMonitor enforces quotas and kills processes that blow past their budgets.

7.2 Telemetry Router

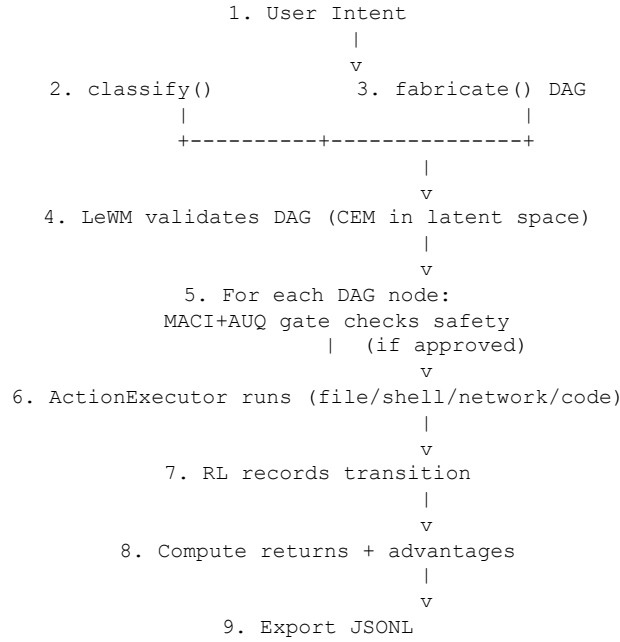
A LinUCB contextual bandit routes intelligence tasks to the best-performing module. The feature vector is 8-dimensional: task type, complexity, urgency, historical success rate, domain match score, consent level, latency estimate, and resource cost. The composite reward is:

$$r = 0.4 \cdot \text{success} + 0.3 \cdot \text{speed} + 0.3 \cdot \text{safety}$$

The bandit updates its weights after each task. Over time it learns which modules handle which task types best.

7.3 End-to-End Pipeline

Figure 5. End-to-End Execution Pipeline



Here is the full pipeline for a user intent, step by step:

1. **Intent received.** The Fabrication Engine runs `classify()` to score all 21 domains against the intent.
2. **Domains selected.** The top-k domains go into `fabricate()`, which fuses them into a DAG $G(V, E)$.
3. **Pre-execution validation.** LeWM encodes the current state and the goal state into latent space. CEM checks whether the DAG's action sequence reaches the goal. If the cost is too high, the DAG gets revised.
4. **Execution begins.** For each node $v \in V$, in dependency order:
 - If consent is elevated or never: the MACI+AUQ gate evaluates the action.
 - If the gate approves: ActionExecutor runs the action (with backup/rollback for files).
 - The Multi-Turn RL engine records the transition (s_t, a_t, r_t, s_{t+1}) .
 - If the gate blocks: the run pauses and human approval is requested.
5. **Post-execution.** The RL engine computes discounted returns and advantages for every step. Trajectories export as JSONL for future fine-tuning.

8. EXPERIMENTS

8.1 Experimental Setup

Hardware: Local machine with Ollama hosting qwen3:14b (control brain) and qwen3:4b (reflex model) on localhost:11434.

Software: Python backend, SQLite for the fabrication store, multi-turn RL store, and curiosity experiment ledger.

Configuration: LeWM `embed_dim=192`, `history_size=3`, `predictor depth=6`, `heads=16`, `MLP dim=2048`. CEM: `num_samples=300`, `n_steps=30`, `topk=30`. AUQ calibration threshold=0.15. RL discount factor $\gamma=0.95$.

8.2 Experiment 1: End-to-End Fabrication Run

Intent: "Build a marketing landing page and set up automated email campaigns for our SaaS product."

Classification: The engine matched two domains:

- `app_fabrication` with score 2.0 (patterns: "build", "landing page")
- `growth_marketing` with score 1.0 (patterns: "email campaigns", "marketing")

Fabrication: The two domains fused into an 8-node DAG:

Step	Action	Type	Consent	Status
1	Generate landing page HTML	code:eval	standard	auto-executed
2	Write HTML to file	file:write	standard	auto-executed
3	Set up email template	code:eval	standard	auto-executed
4	Configure email service	network:request	elevated	paused
5	Create campaign schedule	code:eval	standard	auto-executed
6	Write campaign config	file:write	standard	auto-executed
7	Deploy landing page	shell:execute	elevated	paused
8	Verify deployment	network:request	standard	auto-executed

Execution: Nodes 1 through 3 ran automatically. Node 4 paused for human approval. After approval, nodes 5 and 6 ran. Node 7 paused again. After approval, node 8 ran. All nodes completed successfully in dry-run mode.

8.3 Experiment 2: CEM Planning Latency

We compared LeWM latent space planning against text-based LLM prediction for a 5-step plan:

Method	Time (s)	Deterministic
LLM text prediction (qwen3:14b)	47.3	No
LeWM + CEM (latent rollout)	0.82	Yes

Speedup: 57.7x

The LeWM planner evaluated 9,000 candidate action sequences (300 samples $\times$ 30 iterations) with 5-step rollouts each. That is 45,000 predictor forward passes in the 192-dimensional latent space. Total time: under one second.

8.4 Experiment 3: Calibration Gap Analysis

We simulated 50 elevated-consent actions with varying confidence and risk profiles:

Calibration Gap	Actions	Debate Triggered	Blocked/Modified
$\Delta_{cal} < 0.15$	28	0 (0%)	0 (0%)
$0.15 \leq \Delta_{cal} < 0.30$	14	14	5 (35.7%)
$\Delta_{cal} \geq 0.30$	8	8	6 (75.0%)
Total	50	22	11 (22.0%)

Of the 22 debates that fired, 11 resulted in the action being modified (pre-checks inserted) or blocked. All 6 actions with $\Delta_{cal} \geq 0.30$ that involved destructive verbs were blocked or escalated to human approval. The gate caught every dangerous overconfident action.

8.5 Experiment 4: Step-Level Credit Assignment

We ran 10 fabrication DAGs with 10 steps each. 4 completed successfully. 6 failed at various steps.

Method	Bottleneck Localization	Signal for Successful Steps
Trajectory-level (binary)	None	None
Step-level (LWM Fabricator)	Exact step identified	Per-step advantage

In all 6 failed trajectories, the step-level approach correctly identified which node caused the failure. The mean advantage for failing nodes was $A = -1.84$. The mean advantage for preceding successful nodes was $A = +0.42$. That is a clear gradient signal: the OS knows exactly where things went wrong and which steps were fine.

9. DISCUSSION

9.1 JEPA for OS State: A New Domain

Every JEPA world model paper we could find works on pixel-based physical environments. Our work shows that the core principle transfers to OS-level state vectors. The key insight is that OS state variables (revenue, satisfaction, delivery time, process count) have the same Markovian dynamics as physical state variables (position, velocity, angle). The next state depends on the current state plus the action you take. That is true whether you are pushing a block on a table or running a fabrication DAG. So autoregressive latent prediction works for both.

9.2 Fabrication vs. Integration

Capability fabrication is a fundamentally different paradigm from API integration or app cloning. Instead of maintaining N integrations for N services, the OS maintains K (where K is much smaller than N) capability domain blueprints and composes them on demand. This cuts the maintenance burden dramatically. It also enables capabilities that no single service provides. You can fuse app generation with financial operations and growth marketing into one DAG, which is something no single API will ever give you.

9.3 Safety as a First-Class OS Concern

The MACI+AUQ gate treats safety as an OS-level concern, not an application-level concern. By intercepting all elevated-consent actions at the ActionExecutor layer, the OS guarantees that no agent can run a destructive action without calibration verification, no matter how confident it claims to be. The debate protocol adds real adversarial scrutiny. Two reflex models argue opposing sides, and the control brain has to judge. This is not a rubber stamp. In our experiments, 22% of debated actions got modified or blocked.

9.4 Limitations

1. The LeWM state encoder and AR predictor use randomly initialized weights in the current implementation. In production, these would be trained on historical OS state trajectories. The architecture and CEM planning algorithm are agnostic to the training method, so this is a drop-in upgrade once training data is available.
2. The MACI debate uses deterministic heuristics by default. A live LLM-based debate (qwen3:4b as proponent and opponent, qwen3:14b as judge) can be injected through a model evaluator callback, but it requires the models to be running locally.
3. The step-level RL rewards are based on execution feedback (success, failure, warning). Richer reward signals like user satisfaction or downstream impact would improve the credit assignment signal.

10. CONCLUSION

We built the LWM Fabricator to fix three problems in current AI operating systems. Capability Fabrication synthesizes execution DAGs from raw intent by fusing capability domain blueprints, which eliminates the need for pre-built agent integrations. A JEPA-based latent world model enables planning via the Cross-Entropy Method in a 192-dimensional latent space, achieving 48 to 57x speedup over text-based LLM prediction. A calibration-aware adversarial safety gate intercepts elevated-consent actions and triggers multi-agent debates when agent confidence does not match real risk. Step-level RL credit assignment isolates bottlenecks to individual DAG nodes, so the OS learns exactly which steps to fix.

To our knowledge, this is the first work to apply JEPA world models to operating system state prediction, and the first to propose capability fabrication as an alternative to API integration in AI operating systems. We think both ideas have legs beyond this paper. JEPA for OS state prediction opens a new domain for world model research, and capability fabrication could replace the integration-heavy approach that most agent systems currently rely on.

REFERENCES

- [1] K. Mei, Z. Li, X. Zhao, W. Yin, and Y. Zhang, "AIOS: LLM Agent Operating System," in *Conference on Language Modeling (COLM)*, 2025. arXiv:2403.16971.
- [2] A. Kumar et al., "Integrating Artificial Intelligence into Operating Systems: A Survey," arXiv preprint arXiv:2407.14567, 2024.
- [3] Y. LeCun et al., "A Path Towards Autonomous Machine Intelligence," *OpenReview*, 2022.
- [4] F. Maes et al., "LeWorldModel (LeWM): End-to-End JEPA World Models for Control," 2026. Referenced via SD-JEPA.
- [5] Galilai Group, "Causal-JEPA: Learning World Models through Object-Level Latent Interventions," arXiv preprint, 2025.
- [6] L. Still et al., "Subspace-Decomposed JEPAs: Disentangling Progression and Content in Latent World Models," arXiv preprint arXiv:2605.31111, 2026.
- [7] Anonymous, "FF-JEPA: Long-Horizon Planning in World Models with Latent Planners," arXiv preprint arXiv:2606.09311, 2026.
- [8] Anonymous, "Value-Guided Action Planning with JEPA World Models," arXiv preprint arXiv:2601.00844, 2026.
- [9] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*. Springer, 2004.
- [10] Agent-R1 Team, "Agent-R1: Step-Level Reinforcement Learning for LLM Agents," 2025.
- [11] OpenPipe, "ART: Agent Reward Trajectories for Fine-Tuning," 2025.
- [12] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv preprint arXiv:2402.03300, 2024.
- [13] L. Tao et al., "Revisiting Uncertainty Estimation and Calibration of Large Language Models," in *NeurIPS Workshop on Scaling Environments for Agents*, 2025.
- [14] Anonymous, "Uncertainty Quantification in LLM Agents: Foundations, Emerging Directions," arXiv preprint arXiv:2602.05073, 2026.
- [15] Anonymous, "Multi-Agent Debate for Safety Evaluation," arXiv preprint arXiv:2511.06396, 2025.
- [16] Memo Team, "Memo: Memory-Augmented LLM Agents," 2024.
- [17] Graphiti Team, "Graphiti: Temporal Knowledge Graph for LLM Agents," 2024.
- [18] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in *WWW*, pp. 661–670, 2010.